```
  }

  ...
}
```

❶ Side effect storage
❷ Infinite source
❸ Transformation/filtering
❹ Keeps both the left and right materialized values
❺ Waits to allow the stream to produce work
❻ Waits to allow the stream to be canceled

The stream continues to send messages and “store” them in the database for about three seconds, after which it will be `.cancel`led. Then you can prove that the stream is finished when the future is completed.

This section discussed the basic features of Akka Streams that we use in the next two sections. In the next section, we cover a specific case of how to effectively read the journal from the database. This scenario is common when you use the persistence entities covered in chapter 9. The journal is a list of events optimized for writing, not reading. The persistence query is the first step in converting the journal to a readable format. Finally, we cover reading the journal and writing part of it in a new format so you can query the data without having to read the journal each time.

10.2 Akka Persistence Query

Akka provides an interface to read the journal written in chapter 9 using the Akka Persistence module. The journal contains the events of the persistent entity that represent a state change: that is, the event-sourcing part of the entity. With this interface, you can read these events and project them into another store that is optimized to be queried.

The journal is read in a stream fashion—more precisely, in an Akka Streams fashion, with the Akka Persistence Query module. The module’s API provides queries to run against the journal. These are divided into *standard* and *current* queries. Standard queries—those that don’t have `current` in their name—read all events since the beginning of the journal, and current queries read only new events after the query has been executed. All these queries belong to the interface `ReadJournal` (see table 10.1). All queries are infinite streams. Once you connect to the journal database, you can expect events at any time.

Table 10.1 Methods available to consume the journal: inputs and outputs

Interface methods	Input	Output
`eventsByTag` `currentEventsByTag`	`tag: String` `offset: Int`	`List[EventEnvelope]`
`persistenceIds` `currentPersistenceIds`		`List[String]`

`eventsByPersistenceId`
`currentEventsByPersistenceId`
	`id: String`
	`from: Int`
	`to: Int`
		`List[EventEnvelope]`